	SKT Software Solution	CÓDIGO: MI-DES-DEV-002
SKT SOFTWARE SOLUTION	PROCESO: Desarrollo del sistema	VERSIÓN: 1.0
	Backend: Gestión de Catálogos Paramétricos	ELABORACIÓN: 25/04/2026
	Informe técnico de la tarea realizada	ÚLTIMA REVISIÓN: 25/04/2026

1. Título

Backend: Gestión de Catálogos Paramétricos

2. Introducción

La gestión de catálogos paramétricos constituye un componente fundamental dentro del desarrollo de sistemas de información, debido a que permite centralizar datos de uso recurrente y mantener la coherencia entre los distintos módulos de una aplicación. Este tipo de catálogos se emplea para almacenar valores predefinidos como especialidades, tipos de cita, consultorios, estados y otros registros que no deben ser digitados libremente por el usuario, sino controlados por la lógica del sistema.

En el proyecto analizado, el módulo de catálogos fue desarrollado como parte del backend y expuesto mediante una API REST. Su propósito principal consiste en suministrar información estructurada al frontend, especialmente en procesos de reserva y selección de opciones dentro de la plataforma. La implementación fue revisada a nivel de estructura, rutas, lógica de negocio, persistencia y seguridad, identificándose una arquitectura basada en el patrón controlador-servicio de NestJS.

El presente informe expone de manera técnica la forma en que fue implementado dicho módulo, considerando su ubicación en el repositorio, el comportamiento de sus endpoints, las reglas de negocio aplicadas, la interacción con Supabase y las observaciones derivadas del análisis del código fuente. Además, se describen conceptos relevantes que permiten comprender el alcance funcional del componente y su relación con el sistema completo.

3. Objetivo

Describir técnica y académicamente la implementación del backend encargado de la gestión de catálogos paramétricos, detallando su arquitectura, funcionamiento, mecanismos de persistencia, reglas de negocio, consumo desde frontend y aspectos de seguridad, con el fin de documentar la solución desarrollada dentro del proyecto.

4. Desarrollo de la implementación

La implementación del módulo de catálogos paramétricos se encuentra distribuida en varios archivos del backend, lo que evidencia una organización modular orientada a la separación de responsabilidades. El módulo principal se ubica en `backend/src/catalogs/catalogs.module.ts`, mientras que la exposición HTTP se maneja en `backend/src/catalogs/catalogs.controller.ts`. Por su parte, la lógica de negocio se encuentra en `backend/src/catalogs/catalogs.service.ts`, y su registro dentro de la aplicación principal se realiza en `backend/src/app.module.ts`.

El análisis del proyecto muestra que el backend no implementa un repositorio dedicado ni un ORM específico para este módulo. En lugar de ello, el servicio de catálogos realiza consultas directas a Supabase mediante `backend/src/supabase/supabase.service.ts`. Esta decisión técnica simplifica la capa de acceso a datos para los catálogos, aunque también introduce una dependencia directa del comportamiento de Supabase y de sus políticas de acceso.

La estructura general sigue el patrón *Controller -> Service*, ampliamente utilizado en NestJS. El controlador recibe las solicitudes HTTP y delega el procesamiento al servicio, mientras que este último contiene la lógica necesaria para construir la respuesta. Esta separación permite que el código sea más legible, mantenible y fácil de extender en futuras etapas del proyecto.

Dentro del módulo se identifican dos tipos de catálogos. Por un lado, existen catálogos estáticos almacenados en memoria, como especialidades, tipos de cita y consultorios. Por otro lado, el catálogo de doctores se obtiene dinámicamente desde la base de datos, combinando información de las tablas `doctors` y `profiles`. Esta mezcla de fuentes refleja una solución híbrida que prioriza la rapidez de acceso en algunos casos y la persistencia real en otros.

A nivel de seguridad, el controlador fue marcado como público mediante el decorador `@Public()`, lo que implica que los endpoints no requieren autenticación JWT para ser consumidos. Sin embargo, el flujo global de la aplicación sigue incluyendo el guard de autenticación, que evalúa la marca de acceso público antes de permitir la solicitud. Este diseño conserva la coherencia de la seguridad general del sistema sin bloquear recursos que deben estar disponibles para usuarios no autenticados.

5. Arquitectura y funcionamiento

El módulo de catálogos fue diseñado sobre una arquitectura modular y de capas. En términos funcionales, el flujo de ejecución comienza cuando un cliente realiza

una petición HTTP hacia la ruta base `/api/v1/catalogs`. La solicitud llega al controlador, el cual identifica el endpoint requerido y delega la operación al servicio correspondiente.

En el caso del catálogo de doctores, el servicio consulta primero la tabla `doctors` y luego recupera la información asociada de `profiles`. Posteriormente, se realiza una fusión manual de datos en memoria para construir una respuesta unificada. Esta estrategia fue aplicada debido a limitaciones de consulta o de políticas RLS, evitando depender exclusivamente de una unión directa en la base de datos.

El backend cuenta además con un *ValidationPipe* global; sin embargo, en este módulo su impacto es reducido debido a que los endpoints disponibles son de tipo GET y no reciben cuerpo de solicitud. Por esta razón, el módulo no utiliza DTOs de entrada, ya que no existe manipulación de payload por parte del cliente.

El flujo general puede resumirse de la siguiente forma: la solicitud entra por la API, el guard global verifica si el recurso es público, el controlador recibe la petición, el servicio resuelve los datos, y finalmente se devuelve una respuesta JSON estructurada al consumidor.

6. Endpoints implementados

El módulo expone los siguientes endpoints bajo la base `/api/v1/catalogs`:

- **GET /specialties:** devuelve un catálogo estático con objetos del tipo { id, name, description }.
- **GET /appointment-types:** retorna un catálogo estático con la estructura { id, name, durationMinutes }.
- **GET /offices:** entrega un listado estático de consultorios con la estructura { id, name, floor }.
- **GET /doctors:** consulta dinámicamente la base de datos y devuelve registros con información combinada de doctores y perfiles, estructurados como { id, specialty, profiles: { firstName, lastName } }.

Estos endpoints fueron implementados en `backend/src/catalogs/catalogs.controller.ts` y resueltos en `backend/src/catalogs/catalogs.service.ts`. La naturaleza pública de estas rutas permite que sean consumidas por módulos de interfaz sin necesidad de autenticación previa, lo cual es particularmente útil en procesos de reserva de citas.

7. Persistencia y estructura de datos

El análisis del proyecto evidencia que la persistencia asociada al catálogo dinámico se respalda en la base de datos definida en `schema.sql`. Entre las tablas relacionadas se encuentran `profiles` y `doctors`, además de sus respectivas políticas de lectura.

También se identificó la presencia de un seeder en `backend/seed-doctors.ts`, utilizado para poblar registros iniciales, y un script de ajuste de RLS en `backend/fix-rls.ts`. Estos elementos complementan la administración de datos y refuerzan la preparación del entorno de desarrollo o despliegue.

No se observó una carpeta formal de migraciones versionadas para este módulo, lo cual sugiere que el esquema fue manejado mediante scripts o definiciones directas. Desde una perspectiva técnica, esta situación puede facilitar la implementación rápida, aunque también puede limitar el control histórico de cambios estructurales en la base de datos.

8. Reglas de negocio identificadas

A partir del análisis del código fuente y de la estructura del módulo, se identificaron varias reglas de negocio relevantes:

- Las especialidades, tipos de cita y consultorios no se almacenan en tablas, sino en estructuras estáticas dentro del código.
- Los doctores sí provienen de la base de datos y constituyen el único catálogo dinámico del módulo.
- La información de doctores se construye combinando manualmente datos de `doctors` y `profiles`.
- Cuando falta información de perfil, se aplica un valor alternativo o *fallback*, como *Desconocido* o campos vacíos.
- El módulo se expone públicamente y no aplica permisos por rol en estos endpoints específicos.

Estas reglas reflejan una implementación funcional, aunque con ciertos componentes aún dependientes del código fuente, lo cual reduce el nivel de parametrización total esperado en un catálogo completamente administrable desde base de datos.

9. Integración con frontend y otros componentes

El catálogo de doctores no solo existe en el backend, sino que también es consumido por el frontend a través de `frontend/src/app/core/services/catalogs.service.ts`. Asimismo, su uso se extiende al flujo de reserva en `frontend/src/app/portal-paciente/reserva/re` lo que demuestra su relevancia dentro de la experiencia del usuario.

El frontend cuenta con un interceptor en `frontend/src/app/core/interceptors/auth.interce` que agrega el token a solicitudes hacia `/api/v1/*`. A pesar de ello, el endpoint de catálogos permanece accesible sin autenticación debido a su condición pública. Esta interacción evidencia una integración coherente entre la capa de presentación y el backend.

10. Pruebas y validación

Dentro del proyecto se identificó validación automatizada de acceso público en `backend/test/security.e2e-spec.ts`. Esta prueba confirma que el catálogo puede ser consultado sin autenticación cuando el endpoint ha sido declarado como público.

No se encontraron pruebas unitarias específicas para `CatalogsService` o `CatalogsController`, por lo que la cobertura del módulo parece concentrarse principalmente en pruebas de seguridad e integración general. Desde un punto de vista de mantenimiento, esto representa una oportunidad de mejora para reforzar la confiabilidad del comportamiento esperado en cada endpoint.

11. Observaciones técnicas

El análisis del módulo permitió identificar algunos aspectos relevantes desde la perspectiva de arquitectura y mantenimiento:

- Tres de los cuatro catálogos no son verdaderamente paramétricos, ya que permanecen codificados en memoria.
- Los cambios en dichos catálogos requieren modificación y despliegue del backend.
- El manejo de errores en la obtención de doctores utiliza excepciones genéricas, en lugar de excepciones HTTP tipadas de NestJS.

- Se observan registros por consola dentro del servicio, lo cual no resulta ideal para un entorno productivo.
- La obtención de datos de doctores depende de un *merge* manual, lo que refleja una solución práctica ante limitaciones de consulta o políticas de acceso.
- Existe una desalineación parcial entre la documentación técnica y el comportamiento implementado en el código.

Estas observaciones no invalidan la funcionalidad del módulo, pero sí permiten reconocer áreas de mejora para futuras iteraciones del sistema.

12. Definiciones

- **Backend:** capa lógica de una aplicación encargada de procesar solicitudes, aplicar reglas de negocio, administrar datos y responder a los clientes mediante servicios o APIs.
- **Catálogo paramétrico:** conjunto de valores predefinidos que se utilizan para controlar y estandarizar información dentro de un sistema.
- **API REST:** interfaz de comunicación basada en HTTP que permite consultar y administrar recursos de forma estructurada.
- **NestJS:** framework de Node.js orientado a la construcción de aplicaciones escalables y organizadas mediante módulos, controladores y servicios.
- **Supabase:** plataforma backend que ofrece base de datos, autenticación y servicios relacionados para el desarrollo de aplicaciones modernas.
- **Controlador:** componente que recibe solicitudes HTTP y dirige su procesamiento hacia la lógica de negocio correspondiente.
- **Servicio:** capa donde se implementan las reglas de negocio y el procesamiento principal de la información.
- **RLS:** sigla de *Row Level Security*, mecanismo que restringe el acceso a filas específicas de una tabla según políticas definidas.
- **DTO:** objeto de transferencia de datos utilizado para estructurar y validar información entre capas del sistema.

- **Seeder:** script utilizado para insertar datos iniciales en la base de datos.

13. Resultados obtenidos

Como resultado del desarrollo analizado, se obtuvo un módulo backend funcional para la gestión de catálogos, con capacidad de servir información clave para el sistema de reservas y otros componentes de la aplicación. La solución permitió centralizar la consulta de datos, mantener una estructura modular y facilitar el consumo desde el frontend.

Asimismo, el proyecto consolidó la utilización de NestJS como base para la organización del backend y de Supabase como soporte de persistencia. La implementación del catálogo de doctores, en particular, mostró una integración efectiva entre datos almacenados y respuesta estructurada al cliente.

14. Conclusiones

La elaboración del backend para la gestión de catálogos paramétricos permitió resolver una necesidad funcional importante dentro del sistema, al centralizar información de uso recurrente y exponerla mediante una API pública. El enfoque utilizado evidenció una arquitectura clara basada en controladores y servicios, lo que favorece la comprensión y el mantenimiento del código.

También se concluye que, aunque el módulo cumple su función, todavía presenta elementos que pueden fortalecerse, especialmente en la parametrización real de catálogos, la cobertura de pruebas y el manejo formal de errores. Estas oportunidades de mejora representan una base útil para futuras versiones del proyecto.

	SKT Software Solution	CÓDIGO: MI-DES-DEV-002
SKT SOFTWARE SOLUTION	PROCESO: Desarrollo del sistema	VERSIÓN: 1.0
	Backend: Gestión de Catálogos Paramétricos	ELABORACIÓN: 25/04/2026
	Informe técnico de la tarea realizada	ÚLTIMA REVISIÓN: 25/04/2026

15. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		25/04/2026
Desarrollador Backend	Sebastián Ortiz		25/04/2026
Desarrollador Fullstack	Daniel Luisa		25/04/2026
Desarrollador Frontend	Alex Guachi		25/04/2026

16. Control de cambios

Versión	**Descripción del Cambio**	**Responsable del Cambio**	**Fecha de Aprobación**
1.0	Elaboración inicial del informa de la realizacion de la tarea "Backend: Gestión de Catálogos Paramétricos"	Angel Ayuquina	25/04/2026